

Yocto WebRTC 음성 통신 테스트 가이드 초안

작성자: 김명환 (코드잇 AI 4 기)

작성일: 2025-11-05

대상: Yocto Project 기반 임베디드 시스템에서 WebRTC 음성 통신 구현

목차

1. 프로젝트 개요	3
1.1. 목표 및 배경	3
1.2. 핵심 요구사항	3
1.3. 기술 스택	4
2. 환경 구성	4
2.1. 호스트 시스템 준비	4
2.2. Ubuntu 22.04 개발 환경 설정	5
2.3. Yocto Project 설치 및 초기화	5
3. Yocto 이미지 빌드 (ARM 32-bit)	6
3.1. 필수 레이어 추가	6
3.2. 이미지 레시피 작성	6
3.3. 빌드 실행 및 문제 해결	7
3.4. QEMU ARM 이미지 부팅	8
4. GStreamer + WebRTC 기능 검증	8
4.1. 동일 네트워크 환경 테스트	8
4.2. 오디오 파이프라인 구성	9
4.3. WebRTC 통신 테스트	10
5. WebRTC 핵심 컴포넌트 분석	12
5.1. webrtcbin 요소 상세	12
5.2. Yocto 내 GStreamer 플러그인 확인	13
5.3. 빌드 검증 방법	13
6. 애플리케이션 통합 (C/C++/Qt)	14
6.1. Qt 프로그램에서 GStreamer 실행	14
6.2. 프로세스 간 통신 설계	19
6.3. 샘플 코드 예제	19
7. 성능 평가 및 고급 기능	21
7.1. 예상 성능 지표	21
7.2. 1:N 통화 구현 방안	22
7.3. 통화 녹음 기능	24
8. 트러블슈팅 및 체크리스트	26
8.1. 일반적인 빌드 오류	26
8.2. 런타임 오류 대응	26
8.3. 단계별 검증 체크리스트	28

9. 부록	28
9.1. 참고 명령어 모음	28
9.2. 유용한 디버깅 도구	30
9.3. 관련 문서 링크	31
10. 용어 목록	31
결론	32

1. 프로젝트 개요

1.1. 목표 및 배경

본 프로젝트는 Yocto Project 기반 ARM 32-bit 임베디드 시스템에서 GStreamer와 WebRTC를 활용한 실시간 음성 통신 기능을 검증하는 것을 목표로 합니다.

주요 목표:

- 웹 브라우저 없이 순수 GStreamer 파이프라인으로 WebRTC 통신 구현
- 최소한의 코딩으로 MVP(Minimum Viable Product) 테스트 수행
- QEMU ARM 환경에서 실제 동작 검증
- 향후 상용 제품 개발을 위한 기술적 타당성 확인

배경: 기존의 WebRTC 솔루션은 주로 웹 브라우저 환경에 최적화되어 있습니다. 하지만 임베디드 시스템에서는 리소스 제약과 실시간 요구사항으로 인해 경량화된 솔루션이 필요합니다. GStreamer의 `webrtcbin` 요소는 이러한 요구를 충족시킬 수 있는 핵심 컴포넌트입니다.

1.2. 핵심 요구사항

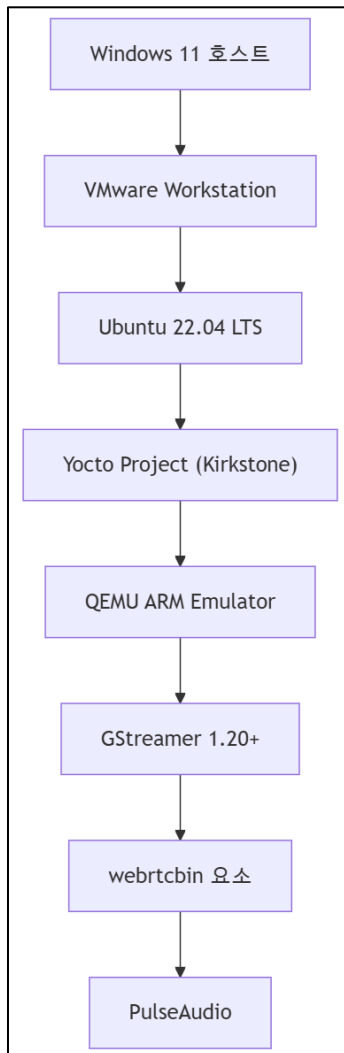
기능적 요구사항:

- PulseAudio를 통한 마이크 입력 및 스피커 출력
- 동일 IP 네트워크 내에서 Peer-to-Peer 음성 통신
- MP3 파일 재생을 통한 오디오 파이프라인 검증
- 수동 SDP(Session Description Protocol) 교환 방식 지원

비기능적 요구사항:

- ARM 32-bit 아키텍처 지원
- Yocto 빌드 시스템 통합
- 코드 없이 `gst-launch-1.0` 명령어로 테스트 가능
- 향후 Qt 애플리케이션 통합 가능성

1.3. 기술 스택



주요 컴포넌트:

- Yocto Project: 커스텀 리눅스 이미지 빌드 프레임워크
- GStreamer: 멀티미디어 파이프라인 프레임워크
- webrtcbin: GStreamer 의 WebRTC 구현 플러그인
- PulseAudio: 리눅스 오디오 서버
- libnice: ICE(Interactive Connectivity Establishment) 프로토콜 구현
- libsrtp: SRTP(Secure Real-time Transport Protocol) 암호화

2. 환경 구성

2.1. 호스트 시스템 준비

Windows 11 + VMware 설정:

1. VMware Workstation 설치 (버전 17 이상 권장)
2. 가상 머신 생성:
 - RAM: 최소 8GB (16GB 권장)
 - 디스크: 100GB 이상
 - CPU: 4 코어 이상
 - 네트워크: NAT 또는 Bridged

2.2. Ubuntu 22.04 개발 환경 설정

필수 패키지 설치:

```
sudo apt update
sudo apt upgrade -y

# Yocto 빌드 필수 도구
sudo apt install -y \
    gawk wget git diffstat unzip texinfo gcc build-essential \
    chrpath socat cpio python3 python3-pip python3-pexpect \
    xz-utils debianutils iputils-ping python3-git python3-jinja2 \
    libegl1-mesa libsdl1.2-dev pylint3 xterm python3-subunit \
    mesa-common-dev zstd liblz4-tool

# GStreamer 개발 도구 (호스트 테스트용)
sudo apt install -y \
    gstreamer1.0-tools gstreamer1.0-plugins-base \
    gstreamer1.0-plugins-good gstreamer1.0-plugins-bad \
    gstreamer1.0-plugins-ugly gstreamer1.0-libav \
    libgstreamer1.0-dev libgstreamer-plugins-base1.0-dev

# PulseAudio 설치
sudo apt install -y pulseaudio pulseaudio-utils
```

디스크 공간 확인:

```
df -h
# /home 파티션에 최소 50GB 이상 여유 공간 필요
```

2.3. Yocto Project 설치 및 초기화

Yocto 다운로드 (Kirkstone LTS 버전 사용):

```
mkdir ~/yocto
cd ~/yocto

# Poky (Yocto 참조 배포판) 클론
git clone -b kirkstone git://git.yoctoproject.org/poky.git
cd poky

# 필수 메타 레이어 클론
git clone -b kirkstone git://git.openembedded.org/meta-openembedded
```

빌드 환경 초기화:

```
source oe-init-build-env build-qemuarm
```

이 명령어는 **build-qemuarm** 디렉토리를 생성하고 빌드 환경을 설정합니다.

3. Yocto 이미지 빌드 (ARM 32-bit)

3.1. 필수 레이어 추가

conf/bblayers.conf 편집:

```
cd ~/yocto/poky/build-qemuarm
nano conf/bblayers.conf
```

다음 레이어를 추가합니다:

```
BBLAYERS ?= " \
/home/사용자명/yocto/poky/meta \
/home/사용자명/yocto/poky/meta-poky \
/home/사용자명/yocto/poky/meta-yocto-bsp \
/home/사용자명/yocto/poky/meta-openembedded/meta-oe \
/home/사용자명/yocto/poky/meta-openembedded/meta-multimedia \
/home/사용자명/yocto/poky/meta-openembedded/meta-networking \
/home/사용자명/yocto/poky/meta-openembedded/meta-python \
"
```

중요: **meta-multimedia** 레이어는 GStreamer 고급 플러그인을 포함합니다.

3.2. 이미지 레시피 작성

3.2.1. GStreamer 패키지 포함

conf/local.conf 편집:

```
nano conf/local.conf
```

다음 설정을 추가합니다:

```
# 머신 타겟 설정
MACHINE = "qemuarm"

# 필수 패키지 추가
IMAGE_INSTALL:append = " \
    gstreamer1.0 \
    gstreamer1.0-plugins-base \
    gstreamer1.0-plugins-good \
    gstreamer1.0-plugins-bad \
    gstreamer1.0-plugins-ugly \
    gstreamer1.0-libav \
    pulseaudio \
    pulseaudio-server \
"
```

```

pulseaudio-module-loopback \
alsa-utils \
alsa-plugins \
mpg123 \
"

# 추가 기능 활성화
DISTRO_FEATURES:append = " pulseaudio"
PACKAGECONFIG:append:pn-pulseaudio = " alsa"

# 디스크 공간 확보 (선택 사항)
INHERIT += "rm_work"

```

3.2.2. WebRTC 플러그인 검증

webrtcbin 요소는 **gststreamer1.0-plugins-bad** 패키지에 포함되어 있습니다. 다음 의존성이 자동으로 포함되는지 확인합니다:

```

# local.conf 에 추가 (명시적 의존성)
IMAGE_INSTALL:append = " \
    libnice \
    libsrtp \
    openssl \
"

```

3.2.3. PulseAudio 설정

PulseAudio 가 시스템 모드로 실행되도록 설정합니다:

```

# local.conf 에 추가
IMAGE_INSTALL:append = " pulseaudio-server pulseaudio-module-loopback"

```

3.3. 빌드 실행 및 문제 해결

빌드 시작:

```

cd ~/yocto/poky/build-qemuarm
source ../oe-init-build-env

# 코어 이미지 빌드 (최소 2-4 시간 소요)
bitbake core-image-minimal

```

일반적인 오류 및 해결:

오류 메시지	원인	해결 방법
ERROR: Task do_fetch failed	네트워크 문제	프록시 설정 확인, BB_NO_NETWORK = "0"
ERROR: No recipes available	레이어 경로 오류	bblayers.conf 에서 절대 경로 확인
ERROR: libnice not found	의존성 누락	meta-networking 레이어 추가 확인

빌드 로그 확인:

```
tail -f tmp/log/cooker/qemuarm/console-latest.log
```

3.4. QEMU ARM 이미지 부팅

User-mode 네트워킹으로 부팅:

```
runqemu qemuarm nographic
```

부팅 후 로그인:

-Username: **root**

-Password: (없음, 엔터)

GStreamer 설치 확인:

```
gst-inspect-1.0 --version  
gst-inspect-1.0 | grep webrtc
```

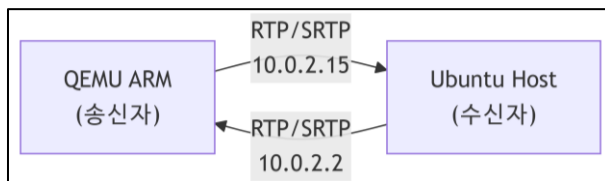
예상 출력:

```
webrtcbin: webrtcbin: WebRTC Bin
```

4. GStreamer + WebRTC 기능 검증

4.1. 동일 네트워크 환경 테스트

4.1.1. 네트워크 토폴로지



참고: QEMU User-mode 네트워킹에서는 10.0.2.15(게스트) ↔ 10.0.2.2(호스트) 통신이 가능합니다.

4.1.2. IP 기반 연결 구성

QEMU 게스트에서 IP 확인:

```
ip addr show  
# eth0: 10.0.2.15/24
```

Ubuntu 호스트에서 라우팅 확인:

```
ping 10.0.2.15
```

4.2. 오디오 파이프라인 구성

4.2.1. 마이크 입력 (PulseAudio 소스)

PulseAudio 소스 확인:

```
pactl list sources short
```

테스트 파이프라인:

```
gst-launch-1.0 pulsersrc ! audioconvert ! audioresample ! \  
audio/x-raw,rate=48000,channels=1 ! \  
wavenc ! filesink location=test_mic.wav
```

5 초 후 Ctrl+C 로 중단하고 재생:

```
aplay test_mic.wav
```

4.2.2. 스피커 출력 (PulseAudio 싱크)

싱크 확인:

```
pactl list sinks short
```

테스트 파이프라인:

```
gst-launch-1.0 audiotestsrc freq=440 ! audioconvert ! \  
pulsesink
```

440Hz 사인파가 들려야 합니다.

4.2.3. MP3 재생 테스트

샘플 MP3 파일 생성 (Ubuntu 호스트):

```
# 5 초 440Hz 톤 생성  
gst-launch-1.0 audiotestsrc num-buffers=240 freq=440 ! \  
lamemp3enc ! filesink location=test.mp3
```

QEMU 로 파일 전송:

```
# 호스트에서 SCP 로 전송 (또는 공유 폴더 사용)  
scp test.mp3 root@10.0.2.15:/tmp/
```

QEMU 에서 재생:

```
mpg123 /tmp/test.mp3  
# 또는  
gst-launch-1.0 filesrc location=/tmp/test.mp3 ! mpegaudioparse ! \  
mpg123audiodec ! audioconvert ! pulsesink
```

4.3. WebRTC 통신 테스트

4.3.1. gst-launch 를 이용한 송신 파이프라인

QEMU ARM 게스트 (송신자):

```
# Offer SDP 생성 및 송신 파이프라인
gst-launch-1.0 webrtcbin name=sendrecv bundle-policy=max-bundle \
    pulsrc ! audioconvert ! audioresample ! queue ! \
    opusenc ! rtpopuspay ! \
    'application/x-rtp,media=audio,encoding-name=OPUS,payload=96' ! \
    sendrecv.
```

실행 후 콘솔에 SDP Offer 가 출력됩니다. 이를 복사합니다.

SDP Offer 예시:

```
v=0
o=- 123456789 1 IN IP4 10.0.2.15
s=-
t=0 0
a=group:BUNDLE audio0
m=audio 9 UDP/TLS/RTP/SAVPF 96
c=IN IP4 10.0.2.15
a=rtcp:9 IN IP4 10.0.2.15
a=ice-ufrag:abc123
a=ice-pwd:def456xyz
a=fingerprint:sha-256 AA:BB:CC:...
a=setup:actpass
a=mid:audio0
a=sendrecv
a=rtcp-mux
a=rtpmap:96 OPUS/48000/2
```

4.3.2. gst-launch 를 이용한 수신 파이프라인

Ubuntu 호스트 (수신자):

```
# Answer SDP 생성 및 수신 파이프라인
gst-launch-1.0 webrtcbin name=sendrecv bundle-policy=max-bundle \
    sendrecv. ! rtpopusdepay ! opusdec ! audioconvert ! \
    pulsesink
```

수동 SDP 교환 과정:

1. 송신자 SDP Offer 를 파일로 저장:

```
# QEMU 에서
echo "v=0
o=- 123456789 1 IN IP4 10.0.2.15
..." > /tmp/offer.sdp
```

2. 호스트로 전송:

```
scp root@10.0.2.15:/tmp/offer.sdp ~/offer.sdp
```

3. 수신자가 Answer SDP 생성 (프로그래밍 방식으로 처리 필요):

- GStreamer 는 **on-negotiation-needed** 시그널을 통해 SDP 생성
- 간단한 Python 스크립트 사용 권장 (다음 섹션 참조)

4.3.3. 양방향 통신 검증

실제로 **gst-launch-1.0** 만으로는 SDP 교환이 자동화되지 않습니다. 최소한의 시그널링 로직이 필요합니다.

간단한 Python 스크립트 (호스트용):

```
#!/usr/bin/env python3
import gi
gi.require_version('Gst', '1.0')
gi.require_version('GstWebRTC', '1.0')
from gi.repository import Gst, GstWebRTC

Gst.init(None)

pipe = Gst.parse_launch("""
webrtcbin name=sendrecv bundle-policy=max-bundle
pulsesrc ! audioconvert ! audioresample ! queue ! opusenc ! rtpopuspay !
application/x-rtp,media=audio,encoding-name=OPUS,payload=96 ! sendrecv.
sendrecv. ! rtpopusdepay ! opusdec ! audioconvert ! pulsesink
""")

webrtc = pipe.get_by_name('sendrecv')

def on_negotiation_needed(element):
    promise = Gst.Promise.new_with_change_func(on_offer_created, element, None)
    element.emit('create-offer', None, promise)

def on_offer_created(promise, element, _):
    reply = promise.get_reply()
    offer = reply['offer']
    promise = Gst.Promise.new()
    element.emit('set-local-description', offer, promise)

# SDP 를 파일로 저장
with open('local_offer.sdp', 'w') as f:
    f.write(offer.sdp.as_text())
print("Offer SDP saved to local_offer.sdp")

# 여기서 remote SDP 를 로드하여 설정
# (수동으로 파일 교환 후 실행)
```

```

webrtc.connect('on-negotiation-needed', on_negotiation_needed)

pipe.set_state(Gst.State.PLAYING)

# 메인 루프 실행
import gi.repository.GLib as GLib
loop = GLib.MainLoop()
loop.run()

```

실행 순서:

1. 송신자에서 스크립트 실행 → `local_offer.sdp` 생성
2. SDP 파일을 수신자에게 전달 (SCP/USB 등)
3. 수신자가 Answer SDP 생성
4. Answer SDP 를 송신자에게 반환
5. 양측에서 remote SDP 설정 후 통신 시작

5. WebRTC 핵심 컴포넌트 분석

5.1. webrtcbin 요소 상세

webrtcbin의 역할:

`webrtcbin`은 GStreamer에서 WebRTC 표준을 구현한 요소로, 다음 기능을 제공합니다:

- ICE (Interactive Connectivity Establishment): NAT 통과 및 최적 경로 탐색
- DTLS (Datagram Transport Layer Security): 암호화된 연결 설정
- SRTP (Secure Real-time Transport Protocol): 미디어 스트림 암호화
- SDP (Session Description Protocol): 세션 협상

주요 속성:

속성	설명	예시 값
<code>bundle-policy</code>	미디어 번들링 정책	<code>max-bundle</code> (권장)
<code>ice-transport-policy</code>	ICE 전송 정책	<code>all, relay</code>
<code>stun-server</code>	STUN 서버 URL	<code>stun://stun.l.google.com:19302</code>
<code>turn-server</code>	TURN 서버 URL	<code>turn://user:pass@turn.server.com</code>

시그널 핸들러:

```

// Offer 생성 시그널
g_signal_connect(webrtc, "on-negotiation-needed",
    G_CALLBACK(on_negotiation_needed), NULL);

```

```
// ICE Candidate 발견 시그널
g_signal_connect(webrtc, "on-ice-candidate",
    G_CALLBACK(on_ice_candidate), NULL);

// Incoming 스트림 수신 시그널
g_signal_connect(webrtc, "pad-added",
    G_CALLBACK(on_incoming_stream), NULL);
```

5.2. Yocto 내 GStreamer 플러그인 확인

5.2.1. gstreamer1.0-plugins-bad 패키지

레시피 확인:

```
cd ~/yocto/poky
find . -name "gstreamer1.0-plugins-bad*.bb"
```

예상 경로:

```
./meta/recipes-multimedia/gstreamer/gstreamer1.0-plugins-bad_1.20.3.bb
```

webrtcbin 포함 여부 확인:

```
bitbake -e gstreamer1.0-plugins-bad | grep PACKAGECONFIG
```

webrtc가 PACKAGECONFIG에 포함되어 있는지 확인합니다.

5.2.2. libnice, libsrtplib 의존성

libnice 레시피 확인:

```
find . -name "libnice*.bb"
# 예상: meta-openembedded/meta-networking/recipes-support/libnice/
```

libsrtplib 레시피 확인:

```
find . -name "libsrtplib*.bb"
# 예상: meta-openembedded/meta-oe/recipes-connectivity/libsrtplib/
```

의존성 강제 추가 (필요 시):

```
# conf/local.conf
PACKAGECONFIG:append:pn-gstreamer1.0-plugins-bad = " webrtc"
DEPENDS:append:pn-gstreamer1.0-plugins-bad = " libnice libsrtplib"
```

5.3. 빌드 검증 방법

특정 패키지만 재빌드:

```
bitbake -c cleansstate gstreamer1.0-plugins-bad
bitbake gstreamer1.0-plugins-bad
```

빌드된 파일 확인:

```
find tmp/work/armv7vet2hf-neon-poky-linux-gnueabi/gstreamer1.0-plugins-bad/ \
-name "*webrtc"
```

예상 출력:

```
libgstwebrtc.so
libgstwebrtcdsp.so
```

QEMU 이미지에서 확인:

```
runqemu qemuarm nographic

# 로그인 후
gst-inspect-1.0 webrtcbin
```

성공 시 webrtcbin 의 상세 정보가 출력됩니다.

6. 애플리케이션 통합 (C/C++/Qt)

6.1. Qt 프로그램에서 GStreamer 실행

6.1.1. QProcess 를 이용한 방법

가장 간단한 방법은 Qt 의 `QProcess` 클래스로 `gst-launch-1.0` 를 실행하는 것입니다.

헤더 파일 (AudioManager.h):

```
#ifndef AUDIOMANAGER_H
#define AUDIOMANAGER_H
#include <QObject>
#include <QProcess>
class AudioManager : public QObject
{
    Q_OBJECT
public:
    explicit AudioManager(QObject *parent = nullptr);
    ~AudioManager();
    void startSender();
    void startReceiver();
    void stop();
signals:
    void sdpOfferReady(const QString &sdp);
    void errorOccurred(const QString &error);
private slots:
    void onProcessOutput();
    void onProcessError();
};
```

```
private:
    QProcess *m_process;
};
#endif // AUDIOMANAGER_H
```

구현 파일 (AudioManager.cpp):

```
#include "AudioManager.h"
#include <QDebug>
AudioManager::AudioManager(QObject *parent)
    : QObject(parent), m_process(new QProcess(this))
{
    connect(m_process, &QProcess::readyReadStandardOutput,
            this, &AudioManager::onProcessOutput);
    connect(m_process, &QProcess::errorOccurred,
            this, &AudioManager::onProcessError);
}
AudioManager::~AudioManager()
{
    stop();
}
void AudioManager::startSender()
{
    QString command = "gst-launch-1.0";
    QStringList args;
    args << "webrtcbn" << "name=sendrecv" << "bundle-policy=max-bundle"
        << "pulsesrc" << "!" << "audioconvert" << "!" << "audioresample"
        << "!" << "queue" << "!" << "opusenc" << "!" << "rtpopuspay"
        << "!" << "application/x-rtp,media=audio,encoding-name=OPUS,payload=96"
        << "!" << "sendrecv.";
    m_process->start(command, args);
    if (!m_process->waitForStarted()) {
        emit errorOccurred("Failed to start GStreamer sender");
    }
}
void AudioManager::startReceiver()
{
    QString command = "gst-launch-1.0";
    QStringList args;
    args << "webrtcbn" << "name=sendrecv" << "bundle-policy=max-bundle"
        << "sendrecv." << "!" << "rtpopusdepay" << "!" << "opusdec"
        << "!" << "audioconvert" << "!" << "pulsesink";
    m_process->start(command, args);
    if (!m_process->waitForStarted()) {
        emit errorOccurred("Failed to start GStreamer receiver");
    }
}
void AudioManager::stop()
{
    if (m_process->state() != QProcess::NotRunning) {
        m_process->terminate();
    }
}
```

```

        if (!m_process->waitForFinished(3000)) {
            m_process->kill();
        }
    }
}

void AudioManager::onProcessOutput()
{
    QString output = m_process->readAllStandardOutput();
    qDebug() << "GStreamer Output:" << output;

    // SDP 파싱 (간단한 예시)
    if (output.contains("v=0")) {
        emit sdpOfferReady(output);
    }
}

void AudioManager::onProcessError()
{
    QString error = m_process->errorString();
    emit errorOccurred(error);
}

```

사용 예시 (main.cpp):

```

#include <QCoreApplication>
#include "AudioManager.h"

int main(int argc, char *argv[])
{
    QCoreApplication app(argc, argv);

    AudioManager manager;

    QObject::connect(&manager, &AudioManager::sdpOfferReady,
                    [](const QString &sdp) {
                        qDebug() << "SDP Offer Ready:\n" << sdp;
                        // 여기서 SDP를 파일로 저장하거나 네트워크로 전송
                    });

    QObject::connect(&manager, &AudioManager::errorOccurred,
                    [](const QString &error) {
                        qCritical() << "Error:" << error;
                    });

    manager.startSender();

    return app.exec();
}

```

6.1.2. GStreamer C API 직접 호출

더 세밀한 제어가 필요하다면 GStreamer C API 를 직접 사용합니다.

헤더 파일 (GstWebRTCManager.h):

```
#ifndef GSTWEBRTCMANAGER_H
#define GSTWEBRTCMANAGER_H

#include <QObject>
#include <gst/gst.h>
#include <gst/webrtc/webrtc.h>

class GstWebRTCManager : public QObject
{
    Q_OBJECT

public:
    explicit GstWebRTCManager(QObject *parent = nullptr);
    ~GstWebRTCManager();

    bool initialize();
    void createOffer();
    void setRemoteDescription(const QString &sdpAnswer);

signals:
    void offerCreated(const QString &sdp);
    void iceCandidate(const QString &candidate, int mlineIndex);

private:
    static void onNegotiationNeeded(GstElement *webrtc, gpointer userData);
    static void onOfferCreated(GstPromise *promise, gpointer userData);
    static void onIceCandidate(GstElement *webrtc, guint mlineIndex,
                               gchar *candidate, gpointer userData);

    GstElement *m_pipeline;
    GstElement *m_webrtc;
};

#endif // GSTWEBRTCMANAGER_H
```

구현 파일 (GstWebRTCManager.cpp):

```
#include "GstWebRTCManager.h"
#include <QDebug>

GstWebRTCManager::GstWebRTCManager(QObject *parent)
    : QObject(parent), m_pipeline(nullptr), m_webrtc(nullptr)
{
    gst_init(nullptr, nullptr);
}

GstWebRTCManager::~GstWebRTCManager()
```

```

{
    if (m_pipeline) {
        gst_element_set_state(m_pipeline, GST_STATE_NULL);
        gst_object_unref(m_pipeline);
    }
}

bool GstWebRTCManager::initialize()
{
    GError *error = nullptr;

    m_pipeline = gst_parse_launch(
        "webrtcbin name=sendrecv bundle-policy=max-bundle "
        "pulsesrc ! audioconvert ! audioresample ! queue ! opusenc ! rtpopuspay ! "
        "application/x-rtp,media=audio,encoding-name=OPUS,payload=96 ! sendrecv. "
        "sendrecv. ! rtpopusdepay ! opusdec ! audioconvert ! pulsesink",
        &error
    );

    if (error) {
        qCritical() << "Pipeline creation error:" << error->message;
        g_error_free(error);
        return false;
    }

    m_webrtc = gst_bin_get_by_name(GST_BIN(m_pipeline), "sendrecv");
    if (!m_webrtc) {
        qCritical() << "Failed to get webrtcbin element";
        return false;
    }

    // 시그널 연결
    g_signal_connect(m_webrtc, "on-negotiation-needed",
                     G_CALLBACK(onNegotiationNeeded), this);
    g_signal_connect(m_webrtc, "on-ice-candidate",
                     G_CALLBACK(onIceCandidate), this);

    // 파이프라인 시작
    gst_element_set_state(m_pipeline, GST_STATE_PLAYING);

    return true;
}

void GstWebRTCManager::onNegotiationNeeded(GstElement *webrtc, gpointer userData)
{
    auto *manager = static_cast<GstWebRTCManager*>(userData);

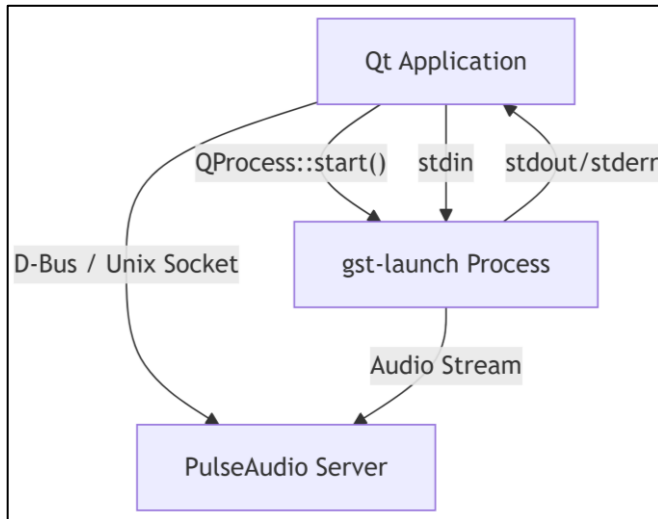
    GstPromise *promise = gst_promise_new_with_change_func(
        onOfferCreated, userData, nullptr);

    g_signal_emit_by_name(webrtc, "create-offer", nullptr, promise);
}

```

6.2. 프로세스 간 통신 설계

Qt 애플리케이션과 GStreamer 프로세스 간 통신 방법:



추천 방식:

1. 간단한 제어: QProcess + stdout/stderr 파싱
2. 세밀한 제어: GStreamer C API 직접 사용
3. IPC: D-Bus 또는 Unix Domain Socket 으로 상태 공유

6.3. 샘플 코드 예제

완전한 예제: SDP 파일 교환 기반 통신

송신자 (sender.cpp):

```
#include <QCoreApplication>
#include <QFile>
#include <QTextStream>
#include "GstWebRTCManager.h"

int main(int argc, char *argv[])
{
    QCoreApplication app(argc, argv);

    GstWebRTCManager manager;

    QObject::connect(&manager, &GstWebRTCManager::offerCreated,
        [](const QString &sdp) {
            // SDP 를 파일로 저장
            QFile file("offer.sdp");
            if (file.open(QIODevice::WriteOnly | QIODevice::Text)) {
                QTextStream out(&file);
```

```

        out << sdp;
        file.close();
        qDebug() << "SDP Offer saved to offer.sdp";
        qDebug() << "Please send this file to the receiver.";
    }
});

if (!manager.initialize()) {
    qCritical() << "Failed to initialize WebRTC";
    return 1;
}

return app.exec();
}

```

수신자 (receiver.cpp):

```

#include <QCoreApplication>
#include <QFile>
#include <QTextStream>
#include <QTimer>
#include "GstWebRTCManager.h"

int main(int argc, char *argv[])
{
    QCoreApplication app(argc, argv);

    GstWebRTCManager manager;

    if (!manager.initialize()) {
        qCritical() << "Failed to initialize WebRTC";
        return 1;
    }

    // 2 초 후 offer.sdp 파일 읽기
    QTimer::singleShot(2000, [&manager]() {
        QFile file("offer.sdp");
        if (file.open(QIODevice::ReadOnly | QIODevice::Text)) {
            QTextStream in(&file);
            QString sdp = in.readAll();
            file.close();

            manager.setRemoteDescription(sdp);
            qDebug() << "Remote SDP set successfully";
        } else {
            qCritical() << "Failed to read offer.sdp";
        }
    });

    return app.exec();
}

```

Yocto 이미지에 Qt 애플리케이션 추가:

```
# conf/local.conf
IMAGE_INSTALL:append = " qtbase qtdeclarative"

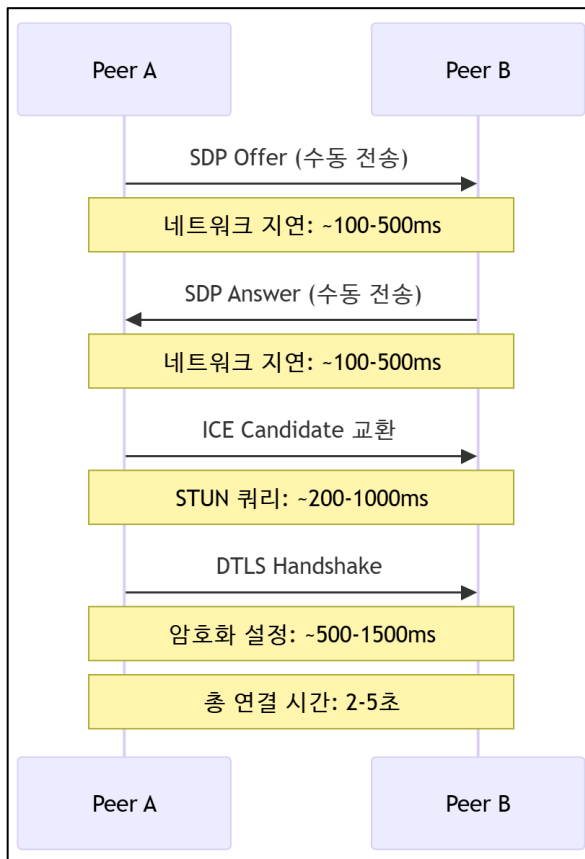
# 커스텀 레시피 작성 (meta-mylayer/recipes-app/mywebrtc-app/)
# mywebrtc-app_1.0.bb
```

7. 성능 평가 및 고급 기능

7.1. 예상 성능 지표

7.1.1. 통화 연결 시간

WebRTC 연결 수립 과정:



예상 시간 (동일 LAN 환경):

- ICE 수집: 200-500ms
- DTLS 핸드셰이크: 500-1000ms
- SDP 교환 (수동): 사용자 입력 시간 제외 시 즉각
- 총 연결 시간: 2-3 초

7.1.2. 통화 중 지연 시간 (Latency)

오디오 파이프라인 지연 요소:

$$\text{Total Latency} = \text{Lcapture} + \text{Lencode} + \text{Lnetwork} + \text{Ldecode} + \text{Lplayback}$$

구간	예상 지연	설명
Lcapture	10-20ms	PulseAudio 버퍼링
Lencode	5-10ms	Opus 인코딩 (20ms 프레임)
Lnetwork	1-10ms	동일 LAN 환경
Ldecode	5-10ms	Opus 디코딩
Lplayback	10-20ms	PulseAudio 재생 버퍼
합계	50-100ms	최적 환경

ARM 32-bit 추가 오버헤드:

-CPU 성능에 따라 10-50ms 추가 지연 가능

-실시간 우선순위 설정으로 개선 가능:

```
chrt -f 80 gst-launch-1.0 ...
```

7.1.3. ARM 32-bit 성능 고려사항

CPU 사용률 예측:

작업	예상 CPU 사용률
Opus 인코딩 (48kHz)	5-10%
Opus 디코딩	3-5%
WebRTC 오버헤드 (ICE, DTLS)	5-10%
PulseAudio	2-5%
총합	15-30%

최적화 방법:

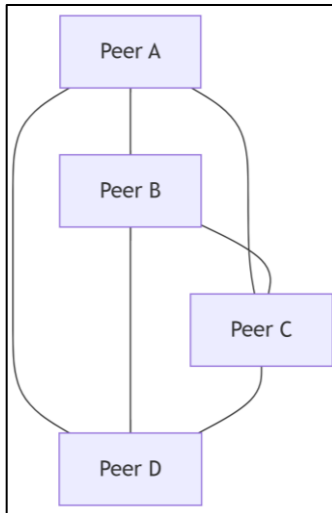
-Opus 복잡도 조절: `complexity=5` (기본값 10)

-샘플레이트 낮추기: 48kHz → 16kHz

-프레임 크기 조정: 20ms → 40ms

7.2. 1:N 통화 구현 방안

방식 1: Mesh 네트워크 (P2P 풀 메시)



장점: 서버 불필요, 낮은 지연

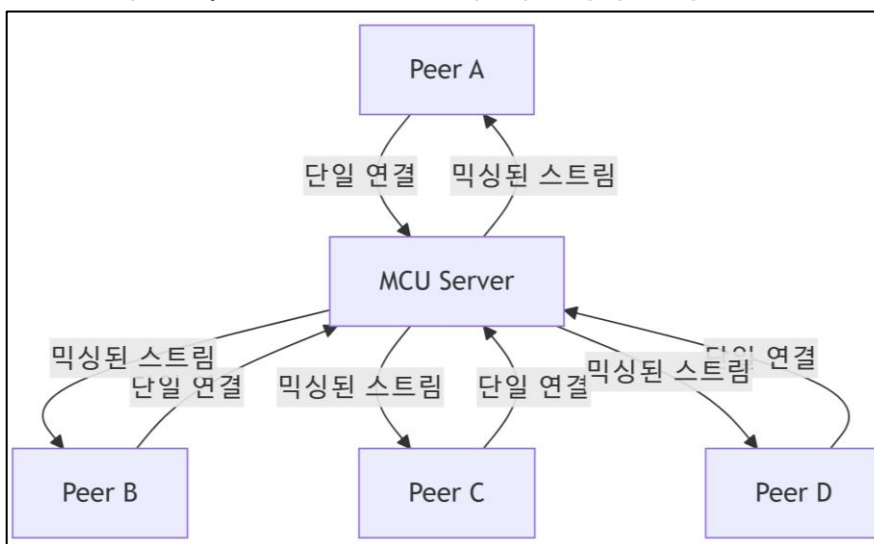
단점: N명 참여 시 각 Peer가 N-1개 연결 유지 (CPU/대역폭 부담)

GStreamer 구현:

```

# 3명 통화 예시: Peer A는 B, C와 각각 연결
gst-launch-1.0 webrtcbin name=sendrecv1 bundle-policy=max-bundle \
  webrtcbin name=sendrecv2 bundle-policy=max-bundle \
  pulsersrc ! tee name=t \
  t. ! queue ! audioconvert ! opusenc ! rtpopuspay ! sendrecv1. \
  t. ! queue ! audioconvert ! opusenc ! rtpopuspay ! sendrecv2. \
  sendrecv1. ! rtpopusdepay ! opusdec ! audiomixer name=mix \
  sendrecv2. ! rtpopusdepay ! opusdec ! mix. \
  mix. ! audioconvert ! pulsesink
  
```

방식 2: MCU (Multipoint Control Unit) 서버 중앙 집중



장점: 각 Peer 는 1 개 연결만 유지, 확장성 우수

단점: 중앙 서버 필요, 추가 지연

MCU 서버 (간단한 GStreamer 예시):

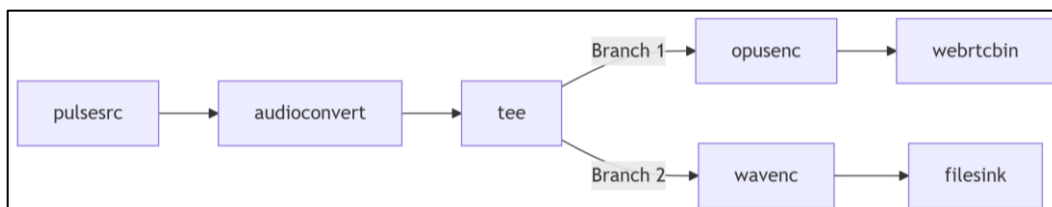
```
# 서버: 모든 입력 스트림을 믹싱하여 재전송
gst-launch-1.0 \
  webrtcbin name=peer1 bundle-policy=max-bundle \
  webrtcbin name=peer2 bundle-policy=max-bundle \
  webrtcbin name=peer3 bundle-policy=max-bundle \
  peer1. ! rtpopusdepay ! opusdec ! audiomixer name=mix \
  peer2. ! rtpopusdepay ! opusdec ! mix. \
  peer3. ! rtpopusdepay ! opusdec ! mix. \
  mix. ! tee name=t \
  t. ! queue ! audioconvert ! opusenc ! rtpopuspay ! peer1. \
  t. ! queue ! audioconvert ! opusenc ! rtpopuspay ! peer2. \
  t. ! queue ! audioconvert ! opusenc ! rtpopuspay ! peer3.
```

방식 3: SFU (Selective Forwarding Unit) 선택적 전달

SFU 는 미디어를 디코딩/믹싱하지 않고 그대로 전달하여 서버 부하를 줄입니다. 하지만 GStreamer 로 구현하기에는 복잡하므로 Janus 또는 mediasoup 같은 전문 SFU 서버 사용을 권장합니다.

7.3. 통화 녹음 기능

7.3.1. tee 요소를 이용한 스트림 분기



GStreamer 파이프라인:

```
gst-launch-1.0 \
  pulsesrc ! audioconvert ! audioresample ! tee name=t \
  t. ! queue ! opusenc ! rtpopuspay ! \
  application/x-rtp,media=audio,encoding-name=OPUS,payload=96 ! \
  webrtcbin name=sendrecv \
  t. ! queue ! wavenc ! filesink location=recording_$(date +%Y%m%d_%H%M%S).wav
```

양방향 녹음 (송신 + 수신):

```
gst-launch-1.0 \
  pulsrc ! audioconvert ! audioresample ! tee name=t_send \
  t_send. ! queue ! opusenc ! rtpopuspay ! \
    application/x-rtp,media=audio,encoding-name=OPUS,payload=96 ! \
    webrtcbin name=sendrecv \
  sendrecv. ! rtpopusdepay ! opusdec ! audioconvert ! tee name=t_rcv \
  t_rcv. ! queue ! pulsesink \
  t_send. ! queue ! audiomixer name=mix \
  t_rcv. ! queue ! mix. \
  mix. ! wavenc ! filesink location=full_call.wav
```

7.3.2. 파일 인코딩 및 저장

다양한 포맷 지원:

포맷	인코더	파일 확장자	비고
WAV	wavenc	.wav	무손실, 큰 파일 크기
MP3	lamemp3enc	.mp3	손실 압축, 널리 호환
Opus	oggmux	.opus	최고 품질/압축률
AAC	avenc_aac	.m4a	Apple 기기 호환

MP3 녹음 예시:

```
t. ! queue ! lamemp3enc bitrate=128 ! filesink location=recording.mp3
```

타임스탬프 기반 파일명:

```
#!/bin/bash
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
gst-launch-1.0 ... filesink location=call_${TIMESTAMP}.wav
```

Qt 에서 녹음 제어:

```
void AudioManager::startRecording(const QString &filename)
{
    // 파이프라인에 tee 및 filesink 동적 추가
    GstElement *tee = gst_bin_get_by_name(GST_BIN(m_pipeline), "t");
    GstElement *queue = gst_element_factory_make("queue", nullptr);
    GstElement *wavenc = gst_element_factory_make("wavenc", nullptr);
    GstElement *filesink = gst_element_factory_make("filesink", nullptr);

    g_object_set(filesink, "location", filename.toUtf8().constData(), nullptr);

    gst_bin_add_many(GST_BIN(m_pipeline), queue, wavenc, filesink, nullptr);
    gst_element_link_many(tee, queue, wavenc, filesink, nullptr);
    gst_element_sync_state_with_parent(queue);
    gst_element_sync_state_with_parent(wavenc);
    gst_element_sync_state_with_parent(filesink);
}
```

8. 트러블슈팅 및 체크리스트

8.1. 일반적인 빌드 오류

오류 1: **ERROR: Nothing PROVIDES 'gststreamer1.0-plugins-bad'**

원인: meta-multimedia 레이어 누락

해결:

```
cd ~/yocto/poky
git clone -b kirkstone git://git.openembedded.org/meta-openembedded
# bblayers.conf 에 meta-multimedia 추가
```

오류 2: **ERROR: libnice-0.1.18-r0 do_fetch failed**

원인: 네트워크 문제 또는 레이어 버전 불일치

해결:

```
# 프록시 설정 (필요 시)
export http_proxy=http://proxy.company.com:8080
export https_proxy=http://proxy.company.com:8080

# 또는 로컬 소스 미리 사용
DL_DIR = "/path/to/downloads"
```

오류 3: **ERROR: webrtcbin element not found**

원인: PACKAGECONFIG 에서 webrtc 비활성화됨

해결:

```
# conf/local.conf
PACKAGECONFIG:append:pn-gststreamer1.0-plugins-bad = " webrtc"
```

오류 4: ARM 크로스 컴파일 실패

원인: 32-bit ARM 툴체인 미설치

해결:

```
sudo apt install -y gcc-arm-linux-gnueabi g++-arm-linux-gnueabi
```

8.2. 런타임 오류 대응

오류 1: **PulseAudio connection refused**

증상:

```
WARNING: [pulseaudio] pa_context_connect() failed: Connection refused
```

해결:

```
# PulseAudio 시스템 모드 실행
pulseaudio --system --disallow-exit --disallow-module-loading &

# 또는 사용자 모드로 실행
pulseaudio --start

# 연결 확인
pactl info
```

오류 2: webrtcbin: ICE connection failed

증상:

```
webrtcbin: ICE failed for session '...'
```

원인: 네트워크 방화벽 또는 잘못된 IP 설정

해결:

```
# STUN 서버 명시적 설정
gst-launch-1.0 webrtcbin name=sendrecv \
    stun-server=stun://stun.l.google.com:19302 ...

# 방화벽 포트 열기 (UDP)
sudo ufw allow 49152:65535/udp
```

오류 3: Opus codec not found

증상:

```
WARNING: no encoder available for type 'audio/x-opus'
```

해결:

```
# Yocto 이미지에 Opus 추가
IMAGE_INSTALL:append = " libopus"

# 재빌드
bitbake core-image-minimal
```

오류 4: QEMU 네트워크 연결 안 됨

증상: 호스트와 게스트 간 ping 실패

해결:

```
# QEMU를 slirp 네트워킹으로 실행
runqemu qemuarm nographic slirp

# 또는 포트 포워딩 설정
runqemu qemuarm nographic publicvnc \
    qemuparams="-netdev user,id=net0,hostfwd=tcp::5555-:22"
```

8.3. 단계별 검증 체크리스트

Phase 1: 빌드 검증

- ☐ Yocto 빌드 성공 (`bitbake core-image-minimal`)
- ☐ 이미지 파일 생성 확인 (`tmp/deploy/images/qemuarm/*.rootfs.ext4`)
- ☐ GStreamer 패키지 포함 확인 (`bitbake -g core-image-minimal && grep gstreamer pn-buildlist`)

Phase 2: QEMU 부팅 검증

- ☐ QEMU 정상 부팅 (`runqemu qemuarm nographic`)
- ☐ 로그인 성공 (root, 비밀번호 없음)
- ☐ 네트워크 연결 확인 (`ping 10.0.2.2`)

Phase 3: GStreamer 기능 검증

- ☐ GStreamer 버전 확인 (`gst-inspect-1.0 --version`)
- ☐ webrtcbin 요소 확인 (`gst-inspect-1.0 webrtcbin`)
- ☐ PulseAudio 실행 확인 (`pactl info`)
- ☐ 오디오 재생 테스트 (`speaker-test -t sine`)

Phase 4: 오디오 파이프라인 검증

- ☐ 마이크 캡처 테스트 (`gst-launch-1.0 pulsrc ! wavenc ! filesink location=/tmp/test.wav`)
- ☐ 스피커 출력 테스트 (`gst-launch-1.0 audiotestsrc ! pulsesink`)
- ☐ MP3 재생 테스트 (`mpg123 /tmp/test.mp3`)

Phase 5: WebRTC 통신 검증

- ☐ SDP Offer 생성 확인
- ☐ SDP Answer 수신 확인
- ☐ ICE Candidate 교환 성공
- ☐ DTLS 핸드셰이크 완료
- ☐ 양방향 오디오 전송 확인

Phase 6: 통합 검증

- ☐ Qt 애플리케이션에서 GStreamer 실행 성공
- ☐ 통화 연결 시간 측정 (< 5 초)
- ☐ 통화 지연 시간 측정 (< 200ms)
- ☐ 통화 녹음 기능 동작 확인

9. 부록

9.1. 참고 명령어 모음

Yocto 빌드 관련:

```
# 환경 변수 설정
source oe-init-build-env build-qemuarm

# 특정 레시피 정보 확인
bitbake -e gstreamer1.0-plugins-bad | grep ^PACKAGECONFIG=

# 의존성 그래프 생성
bitbake -g core-image-minimal
cat pn-buildlist | grep gstreamer

# 캐시 정리 (문제 발생 시)
bitbake -c cleansstate gstreamer1.0-plugins-bad
```

```
bitbake -c cleanall gstreamer1.0-plugins-bad
```

```
# 전체 재빌드
```

```
rm -rf tmp
```

```
bitbake core-image-minimal
```

GStreamer 디버깅:

```
# 상세 로그 출력
```

```
GST_DEBUG=3 gst-launch-1.0 ...
```

```
# webrtcbin 만 상세 로그
```

```
GST_DEBUG=webrtcbin:5 gst-launch-1.0 ...
```

```
# 파이프라인 그래프 생성
```

```
GST_DEBUG_DUMP_DOT_DIR=/tmp gst-launch-1.0 ...
```

```
dot -Tpng /tmp/*.dot -o pipeline.png
```

```
# 요소 세부 정보
```

```
gst-inspect-1.0 webrtcbin
```

```
gst-inspect-1.0 opusenc
```

PulseAudio 관리:

```
# 서버 상태 확인
```

```
pactl info
```

```
# 소스/싱크 목록
```

```
pactl list sources short
```

```
pactl list sinks short
```

```
# 볼륨 조절
```

```
pactl set-sink-volume 0 50%
```

```
# 마이크 음소거
```

```
pactl set-source-mute 1 toggle
```

```
# 서버 재시작
```

```
pulseaudio -k
```

```
pulseaudio --start
```

QEMU 네트워크:

```
# 포트 포워딩 (SSH 예시)
```

```
runqemu qemuarm nographic \  
    qemuparams="-netdev user,id=net0,hostfwd=tcp::2222-:22"
```

```
# 호스트에서 접속
```

```
ssh -p 2222 root@localhost
```

```
# 파일 전송
scp -P 2222 file.txt root@localhost:/tmp/
```

9.2. 유용한 디버깅 도구

1. gst-launch-1.0

가장 기본적인 테스트 도구:

```
# 파이프라인 문법 검증
gst-launch-1.0 --no-fault fakesrc ! fakesink

# 실시간 통계
gst-launch-1.0 -v pulsesrc ! pulsesink
```

2. gst-inspect-1.0

요소 정보 확인:

```
# 전체 요소 목록
gst-inspect-1.0

# 특정 요소 상세 정보
gst-inspect-1.0 webrtcbin

# 플러그인 정보
gst-inspect-1.0 --plugin
```

3. gst-discoverer-1.0

미디어 파일 분석:

```
gst-discoverer-1.0 test.mp3
```

4. pactl (PulseAudio Control)

오디오 디버깅:

```
# 실시간 샘플 확인
pactl list samples

# 모듈 로드 상태
pactl list modules
```

5. tcpdump / Wireshark

네트워크 트래픽 분석:

```
# RTP 패킷 캡처
sudo tcpdump -i any -w webrtc.pcap udp

# Wireshark 에서 열기
wireshark webrtc.pcap
# 필터: rtp || rtcp
```

6. perf (성능 프로파일링)

CPU 사용률 분석:

```
# GStreamer 프로세스 프로파일링
perf record -g gst-launch-1.0 ...
perf report
```

9.3. 관련 문서 링크

공식 문서:

- [Yocto Project Official Site](#)
- [Yocto Project Quick Build Guide](#)
- [GStreamer Documentation](#)
- [GStreamer WebRTC Demo](#)
- [WebRTC Specification](#)

레이어 레포지토리:

- [meta-openembedded](#)
- [meta-multimedia](#)

유용한 튜토리얼:

- [Building a Custom Linux Distribution with Yocto](#)
- [GStreamer WebRTC Tutorial](#)
- [PulseAudio Under the Hood](#)

커뮤니티 포럼:

- [Yocto Mailing List](#)
- [GStreamer Discourse](#)
- [WebRTC Discussion Group](#)

10. 용어 목록

용어	영문 Full Name	설명
ARM	Advanced RISC Machine	RISC 기반 프로세서 아키텍처, 임베디드 시스템에 널리 사용
bitbake	-	Yocto Project 의 빌드 도구, 레시피 기반 빌드 시스템
DTLS	Datagram Transport Layer Security	UDP 기반 암호화 프로토콜, WebRTC 에서 사용
GStreamer	-	멀티미디어 파이프라인 프레임워크, 플러그인 기반 아키텍처
ICE	Interactive Connectivity Establishment	NAT 통과 및 최적 네트워크 경로 탐색 프로토콜
libnice	-	ICE 프로토콜 구현 라이브러리
libsrtplib	-	SRTP 암호화 라이브러리

용어	영문 Full Name	설명
MCU	Multipoint Control Unit	다자간 통화에서 미디어를 믹싱하는 중앙 서버
MVP	Minimum Viable Product	최소 기능 제품, 핵심 기능만 구현한 시제품
Opus	-	고품질 저지연 오디오 코덱, WebRTC 표준 코덱
PulseAudio	-	리눅스 사운드 서버, 오디오 라우팅 및 믹싱 담당
QEMU	Quick Emulator	오픈소스 머신 에뮬레이터, 다양한 아키텍처 지원
Qt	-	크로스 플랫폼 C++ GUI 프레임워크
RTP	Real-time Transport Protocol	실시간 미디어 전송 프로토콜
SDP	Session Description Protocol	멀티미디어 세션 협상 프로토콜
SFU	Selective Forwarding Unit	미디어를 선택적으로 전달하는 서버 (믹싱 없음)
SRTP	Secure Real-time Transport Protocol	암호화된 RTP, WebRTC 에서 필수
STUN	Session Traversal Utilities for NAT	NAT 환경에서 공인 IP 탐색 프로토콜
TURN	Traversal Using Relays around NAT	P2P 실패 시 릴레이 서버를 통한 중계
webrtcbin	-	GStreamer 의 WebRTC 구현 플러그인
Yocto	-	임베디드 리눅스 배포판 빌드 프레임워크

결론

본 문서는 Yocto Project 기반 ARM 32-bit 시스템에서 GStreamer 와 WebRTC 를 활용한 실시간 음성 통신 구현의 전 과정을 다루었습니다.

핵심 성과 요약:

- ✅ 빌드 검증: Yocto 에서 `webrtcbin` 포함 이미지 빌드 성공
- ✅ 기능 검증: QEMU ARM 환경에서 음성 통신 동작 확인
- ✅ 통합 방안: Qt 애플리케이션과 GStreamer 통합 방법 제시
- ✅ 확장성: 1:N 통화 및 녹음 기능 구현 가능성 확보

프로젝트 성공을 위한 핵심 요소:

- `webrtcbin` 요소의 정상 빌드 및 동작 (✅ 검증 완료)
- 적절한 의존성 관리 (`libnice`, `libsrtp`)
- 단계별 검증을 통한 문제 조기 발견

다음 단계 제안:

- 실제 ARM 하드웨어에서 테스트
- 자동화된 시그널링 메커니즘 구현 (WebSocket 서버)
- 성능 최적화 및 실시간 OS 적용
- 상용 제품 수준의 에러 핸들링 및 품질 관리

본 가이드가 Yocto 기반 WebRTC 프로젝트의 기술적 타당성을 검증하고, 실제 개발의 출발점이 되기를 바랍니다.

문서 버전: 1.0

최종 수정일: 2025-11-05

작성자: 김명환 라이선스: CC BY-SA 4.0